

SymPy Bug Report

1 Bug 49: Matrix.solve Methods Return Malformed Results on Singular Systems

1.1 Minimal Reproducer

```
from sympy import Matrix

A = Matrix([[1, 1], [2, 2]])
b = Matrix([1, 2])

print("rank(A):", A.rank())
print("rank([A|b]):", A.row_join(b).rank())
print("gauss_jordan_solve:", A.gauss_jordan_solve(b))

for method in ["QR", "LDL", "CRAMER"]:
    try:
        sol = A.solve(b, method=method)
        print(method, sol, sol.shape)
    except Exception as exc:
        print(method, type(exc).__name__, exc)
```

1.2 Observed Behavior

```
SymPy version: 1.14.0
rank(A): 1
rank([A|b]): 1
gauss_jordan_solve: (Matrix([
[1 - tau0],
[   tau0]]), Matrix([[tau0]]))
QR Matrix([[1]]) (1, 1)
LDL Matrix([[nan], [nan]]) (2, 1)
CRAMER Matrix([[nan], [nan]]) (2, 1)
```

The coefficient matrix has two columns, so a solver result for a unique solution would need two entries. The `QR` path instead returns a one-entry matrix, while the `LDL` and `CRAMER` paths return matrices whose entries are `nan`. These are malformed or invalid solver outputs for the system.

1.3 Expected Behavior

For this non-unique singular system, `Matrix.solve` with the explicit methods `QR`, `LDL`, and `CRAMER` should reject the solve, matching the guarded default path, or otherwise return a documented

representation of the full solution set. The proposed regression behavior is to raise `NonInvertibleMatrixError`. The method-specific paths must not return a vector of the wrong dimension and must not return `nan` entries as if they were a solution.

1.4 Mathematical Explanation

The reproduced system is

$$\begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix},$$

or equivalently

$$x + y = 1, \quad 2x + 2y = 2.$$

The second equation is exactly twice the first. Therefore

$$\text{rank}(A) = \text{rank}([A \mid b]) = 1 < 2,$$

so the system is consistent but has infinitely many solutions:

$$(x, y) = (1 - t, t), \quad t \in \mathbb{C}.$$

The `QR` result `[1]` is not merely a different representation of this solution set; it has only one entry for a two-unknown system. The `LDL` and `CRAMER` results `[nan]` also do not satisfy $Ax = b$, since substituting them gives `nan` residuals rather than the zero vector.

1.5 Source-Level Diagnosis

The diagnosis identifies the matrix linear-solver dispatch path. The relevant public entry point and dispatcher are:

```
MatrixBase.solve  sympy/matrices/matrixbase.py
_solve           sympy/matrices/solvers.py
```

The default `GJ/GE` branch uses the Gauss-Jordan solver and checks whether the returned solution contains parameters, which catches this non-unique singular case. In contrast, the method-specific branches for `QR`, `LDL`, and `CRAMER` dispatch directly to `QRsolve`, `LDLsolve`, and `cramer_solve` without the same singularity or rank guard.

The relevant rule is the direct method dispatch after the guarded `GJ/GE` branch:

```
elif method == 'QR':
    return M.QRsolve(rhs)
elif method == 'LDL':
    return M.LDLsolve(rhs)
elif method == 'CRAMER':
    return M.cramer_solve(rhs)
```

This explains the different failure modes. The `QR` implementation back-substitutes through a reduced triangular factor and returns a one-row result for a two-column matrix. The `LDL` path divides through a zero diagonal entry in `D.diagonal_solve`. The `CRAMER` path divides by $\det(A)$, which is zero for this singular matrix. A plausible fix direction supported by the diagnosis is to add a singularity/rank guard, or residual and shape validation, before accepting method-specific solver outputs for singular systems.

1.6 Additional Failing Instances

The independent verification covers the family

$$A(a) = \begin{bmatrix} 1 & 1 \\ a & a \end{bmatrix}, \quad b(a, c) = \begin{bmatrix} c \\ ac \end{bmatrix}.$$

For every listed case, the two equations reduce to the single constraint $x + y = c$. Thus $(c, 0)$ and $(c - 7, 7)$ are both exact solutions, proving that the system is consistent and non-unique. The same solver contract issue appears across the parametrized examples.

Input / Expression	SymPy behavior	Expected behavior
$(a, c) = (2, 1), A = \begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix}, b = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$	QR: $\begin{bmatrix} 1 \end{bmatrix}$ with shape 1×1 ; LDL/CRAMER: $\begin{bmatrix} \text{nan} \\ \text{nan} \end{bmatrix}$	Reject as non-unique, or return the solution set $x + y = 1$.
$(a, c) = (3, 1), A = \begin{bmatrix} 1 & 1 \\ 3 & 3 \end{bmatrix}, b = \begin{bmatrix} 1 \\ 3 \end{bmatrix}$	QR: $\begin{bmatrix} 1 \end{bmatrix}$ with shape 1×1 ; LDL/CRAMER: $\begin{bmatrix} \text{nan} \\ \text{nan} \end{bmatrix}$	Reject as non-unique, or return the solution set $x + y = 1$.
$(a, c) = (2, 5), A = \begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix}, b = \begin{bmatrix} 5 \\ 10 \end{bmatrix}$	QR: $\begin{bmatrix} 5 \end{bmatrix}$ with shape 1×1 ; LDL/CRAMER: $\begin{bmatrix} \text{nan} \\ \text{nan} \end{bmatrix}$	Reject as non-unique, or return the solution set $x + y = 5$.
$(a, c) = (-1, 4), A = \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix},$ $b = \begin{bmatrix} 4 \\ -4 \end{bmatrix}$	QR: $\begin{bmatrix} 4 \end{bmatrix}$ with shape 1×1 ; LDL/CRAMER: $\begin{bmatrix} \text{nan} \\ \text{nan} \end{bmatrix}$	Reject as non-unique, or return the solution set $x + y = 4$.
$(a, c) = (4, -2), A = \begin{bmatrix} 1 & 1 \\ 4 & 4 \end{bmatrix},$ $b = \begin{bmatrix} -2 \\ -8 \end{bmatrix}$	QR: $\begin{bmatrix} -2 \end{bmatrix}$ with shape 1×1 ; LDL/CRAMER: $\begin{bmatrix} \text{nan} \\ \text{nan} \end{bmatrix}$	Reject as non-unique, or return the solution set $x + y = -2$.
$(a, c) = (5, 3), A = \begin{bmatrix} 1 & 1 \\ 5 & 5 \end{bmatrix}, b = \begin{bmatrix} 3 \\ 15 \end{bmatrix}$	QR: $\begin{bmatrix} 3 \end{bmatrix}$ with shape 1×1 ; LDL/CRAMER: $\begin{bmatrix} \text{nan} \\ \text{nan} \end{bmatrix}$	Reject as non-unique, or return the solution set $x + y = 3$.

1.7 Related Issues Assessment

The best-effort deduplication check found SymPy issue #12242 as related public material concerning singular systems, but did not identify it as a covering duplicate of this exact method-specific `Matrix.solve` failure. The deduplication verdict is therefore a new failure mode with no public precedent. The case remains individually actionable because it gives a small reproducible input, identifies three explicit method branches with invalid outputs, and comes with a focused regression test.

1.8 Proposed SymPy Regression Test

```
import pytest

from sympy import Matrix
from sympy.matrices.exceptions import NonInvertibleMatrixError

CASES = [
    (2, 1),
    (3, 1),
    (2, 5),
    (-1, 4),
    (4, -2),
    (5, 3),
]

@pytest.mark.parametrize("a,c", CASES)
@pytest.mark.parametrize("method", ["QR", "LDL", "CRAMER"])
def test_matrix_solve_method_rejects_nonunique_singular_system(a, c, method):
    A = Matrix([[1, 1], [a, a]])
    b = Matrix([c, a*c])

    assert A.rank() == A.row_join(b).rank() == 1
    with pytest.raises(NonInvertibleMatrixError):
        A.solve(b, method=method)
```